

Reporting directives (ISSUE / BUG / TASK) – deep research + design record

Field	Value
Revision	1
Created	2026-07-15
Last modified	2026-07-15T00:00:00Z
Status	active — implemented as §11.4.202
Method	§11.4.8 deep-research-before-implementation; every claim below is a captured FACT read from the tree, never an assumption (§11.4.6)

. Why this document exists

The operator asked for reporting directives (`ISSUE:` / `ISSUE ::` / `/issue` , and the same for `BUG` and `TASK`) that create a proper workable item with the **whole workable-items system automatically up to date and fully in sync** — DB, all documents, and all external trackers — universal, reusable, fully decoupled.

Before writing a line of code, the existing mechanisms were read. **A large part of the requested machinery already existed.** This document records, honestly, **what already existed vs what was genuinely missing** — so the implementation EXTENDS rather than reinvents (§11.4.74 extend-don't-reimplement).

1 1 . What ALREADY existed (FACTS, read from the tree)

. The \$. . action-prefix system – the whole grammar layer

Artefact	State
constitution/actions/registry.yaml	Present. Data-driven: adding an action = adding one row, no code change.
constitution/scripts/action_prefix_lib.sh	Present (895 lines). Parses the prefix, looks it up, expands it. Python path + awk fallback, deliberately byte-identical.
constitution/actions/recognition_instruction.md	Present (rev 3). The canonical LAYER-1 block embedded verbatim in CLAUDE/ AGENTS/QWEN/GEMINI — this is what makes the system work on every CLI agent with zero setup.
constitution/scripts/hooks/action_prefix_expand.sh	Present. The Claude Code User-PromptSubmit LAYER-2 hook.
constitution/scripts/generate_agent_prefix_commands.sh	Present. Generates per-agent slash commands (codex/gemini/qwen) from the registry.
Registered actions	BACKGROUND , REMINDER .

Five grammar forms already worked (all equivalent): NAME :: x , PREFIX::NAME :: x , /NAME x , /PREFIX::NAME x , NAME ---> x — anchored to the first non-blank line, UPPERCASE-only token, \ -escape, stacked prefixes.

. The conflict mechanism the operator asked for – ALREADY PRESENT

Operator requirement: *"Conflicts with existing host/CLI-agent slash commands must be solved by explicit prefix."*

This requirement was already satisfied. The registry rows carry `slash_bare: auto` + `slash_conflicts: [...]`, and §11.4.140 already specifies: a bare `/NAME` is honored **only** when `NAME` does not collide with a host built-in; the namespaced `/PREFIX::NAME` form is **always** unambiguous.

Applied here as FACT: `/bug` is a documented Claude Code built-in (it reports a bug to Anthropic). So `BUG` declares `slash_conflicts: [bug]` — its bare slash form is not honored, while `/DEFAULT::BUG ...` always is, and `BUG: ... /`
`BUG :: ... /` `BUG ---> ...` are unaffected by any host command. `ISSUE` and `TASK` have no documented built-in collision and keep `slash_bare: auto`.

. The workable-items DB SSoT – already complete

`constitution/scripts/workable-items/` — a Go binary + `schema.sql` (9 tables: `items`, `item_history`, `obsolete_details`, `operator_block_details`, `firebase_metadata`, `logic_groups`, `group_paths`, `doc_segments`, `meta`).

Subcommands already implemented: `add` (with

`--type --severity --title --description --id --prefix --created-by`),
`update`, `reopen`, `block`, `close`, `obsolete-details`, `report`,
`validate`, `repair-bodies`, `diff`, `sync md-to-db / db-to-md`, `export`
(Issues + Fixed + both summaries + HTML/PDF/DOCX siblings, tool-gated so a missing pandoc yields an honest message, never a fake sibling).

So item creation and the doc-regeneration engine already existed.

. The sync chain – already existed, project-side

`scripts/testing/workable_items_sync_all.sh` (ATMOSphere) already chains:
§9.2 backup → `repair-bodies` → `validate` → `export` →
¹ `sync_issues_docs.sh` → `docs_chain` (§11.4.106) verify → optional ClickUp push.
It never commits.

. External trackers – the honest state

Tracker	Reality (verified, not assumed)
ClickUp	A generic <code>task_bridge</code> engine exists (<code>tools/task_bridge/</code> , project-side) with a real <code>reconcile</code> command (dry-run + <code>--apply</code>), reading <code>CLICKUP_API_KEY</code> / <code>CLICKUP_LIST_ID</code> / <code>TASK_BRIDGE_DB</code> . Its <code>push</code> / <code>pull</code> / <code>resolve</code> / <code>status</code> / <code>conflicts</code> / <code>init</code> subcommands are stubs .
<code>constitution/submodules/clickup_sync/</code>	Exists but contains docs only (design + research + API reference). No code .
HelixTrack	No client of any kind . The only occurrence anywhere in the tree is an unrelated HelixQA bank filename. There is no bridge, no API base URL, no board id.

¹ ⁶ The memory note that "a ClickUp LIST_ID mapping and a HelixTrack `task_bridge` client may be absent" was **verified true** — reported here rather than assumed.

. Auto-registration on pull

`constitution/scripts/post_update_hook.sh` (§11.4.164) already installs/registers changed skills, MCP configs, hooks and scripts after every constitution pull — so a new registry row + a new engine script reach every consuming project automatically. **No new distribution mechanism was needed**.

. What was genuinely MISSING (and therefore built)

1. **The three actions** `ISSUE` / `BUG` / `TASK` did not exist in the registry.
2. **The** `NAME:` **single-colon form the operator typed did not exist.** The grammar deliberately required `" :: "` (spaces both sides) to avoid matching `key: value` , `Foo::Bar` , and URLs.
3. **No item-creation engine.** Nothing turned a *report* into an item + drove the sync + pushed to trackers. The pieces existed; the composition did not.
4. **No project-agnostic config seam** for reporting (DB path / id prefix / sync command / tracker bindings were nowhere expressed as reusable DATA).
5. **No HelixTrack client** (and no ClickUp *create* path — only reconcile).

. The one real design decision: the single-colon form

Adding `^NAME:` to the grammar naïvely would be a **regression**: ordinary English prose opens with exactly that shape — `NOTE:` , `TODO:` , `WARNING:` , `FIXME:` , `IMPORTANT:` — and §11.4.140 routes an unknown grammar-shaped token to the §11.4.66 **clarify/ASK** path. Every such sentence would become an operator question.

Decision (safe, evidence-based, documented): the single-colon form is **REGISTERED-ACTION-ONLY by construction**. A single-colon token that is not a registered action is a **NO-OP** (an ordinary prompt) — never an ASK, never a subsystem shortcut. The other five forms keep their ASK-on-unknown behaviour, because their shapes (`::` , `--->` , `/`) do not occur in ordinary prose.

This preserves §11.4.6 exactly (an unknown token is still never silently *expanded*; it is simply not a prefix at all) while making `BUG: ...` work.

Proven at runtime (not asserted): `BUG: / ISSUE: / TASK:` → expand; `NOTE: / TODO: / WARNING:` → noop; `\BUG:` → escape; `FOOBAR :: x` → still asks.

. The other real design decision: what **ISSUE** means

§11.4.16 defines a **CLOSED** type set {Bug | Feature | Task}. "Issue" is **not** a type — inventing a fourth would be a violation. So **ISSUE** is a **reporting CHANNEL** that must be **classified** into the closed set:

- Determinable from the report → state it as **FACT** (§11.4.6).
- Ambiguous → **ASK** the operator (§11.4.66 / §11.4.105) before creating.
- Autonomous *and* ambiguous (§11.4.101, asking impossible) → use the §11.4.16-sanctioned **lowest-stakes default Task**, and **record the defaulted classification verbatim in the item** + surface the reclassify command. Never a silent type assertion.

The engine enforces this: `--kind issue` without `--type` is **refused** unless `--autonomous` is passed, in which case the note is persisted in the description.

. Anti-bluff posture (what is proven vs what is honestly pending)

Proven with captured evidence (real DB, real binaries, no mocks — §11.4.27):

- A **BUG** report creates a real row: correct Type/Status/created_by, stable id, the §11.4.148 structured description persisted (DB row count +1).
- The docs are regenerated **from the DB** — the new id appears in `Issues.md`.
- Both trackers **SKIP with honest machine-readable reasons**; no tracker reports PASS without credentials; **no credential value appears in any evidence artefact** (§11.4.10).
- The gate's FUNCTIONAL invariant **sources and runs** the parser (never grep-only, §11.4.108); all 6 paired §1.1 mutations are caught.
- The suite is self-validated (§11.4.107(10)): golden-good, golden-bad (an oracle asserting a never-created id must FAIL), negative-control (prose).

Honestly PENDING (never claimed as done):

- **HelixTrack push: PENDING-OPERATOR-INPUT.** No client exists and the API base URL / token / board id are operator-only secrets (§11.4.10). The engine ships the *seam* (a config-driven tracker entry); with an empty command it SKIPS with `tracker_client_absent` . **A push is never faked.** Tracked as a workable item.
- **ClickUp create-path:** `task_bridge` implements `reconcile (+ --apply )`; its dedicated `push` subcommand is still a stub upstream. Wired through the config seam; a real push requires `CLICKUP_API_KEY` + `CLICKUP_LIST_ID` in the environment, else an honest `credentials_absent` SKIP.
- **Fleet-wide anchor cascade:** the §11.4.202 literal is present in the 4 canonical constitution carriers. The wider ~146-carrier consumer fleet is the same shared, pre-existing cascade backlog that §11.4.199 / §11.4.200 / §11.4.201 are also in (verified: each reports `4 PRESENT, 146 MISSING`) — not a regression introduced here.

. Sources

Read directly from the tree (the authoritative source, §11.4.6): `actions/registry.yaml` , `scripts/action_prefix_lib.sh` , `actions/recognition_instruction.md` , `scripts/generate_agent_prefix_commands.sh` , `scripts/post_update_hook.sh` , `scripts/workable-items/{README.md,schema.sql}` + the binary's own `--help` , `submodules/clickup_sync/docs/**` , `tools/task_bridge/bin/task_bridge --help` , `scripts/testing/workable_items_sync_all.sh` , and the §11.4.140 / §11.4.16 / §11.4.93 / §11.4.148 anchors in `Constitution.md` .

External: the Claude Code built-in slash-command set (`/bug` is a documented built-in) — the basis for the `slash_conflicts: [bug]` declaration.